Lab 1: Cross-Entropy Method (CEM) for CartPole

Introduction

In this lab, we will implement the Cross-Entropy Method (CEM) algorithm to solve the CartPole-v1 environment. CEM is a simple yet effective policy search method that iteratively improves a policy by sampling from a distribution and selecting elite samples.

Student Information

Name: Kevin Cotellesso

Date: 3-12-2026

Part 1: Setup and Environment Testing

Let's start by importing the necessary libraries and creating the CartPole environment.

In [2]:
```
import numpy as np
import gymnasium as gym
import pandas as pd
import matplotlib.pyplot as plt
from IPython import display

# Create environment
env = gym.make("CartPole-v1", render_mode="rgb_array")

total_reward = 0.0
total_steps = 0
obs, info = env.reset()

print(f"Observation space: {env.observation_space}")
print(f"Action space: {env.action_space}")
print(f"Initial observation: {obs}")
```

```
Observation space: Box([-4.8               -inf -0.41887903        -inf],
[4.8               inf 0.41887903        inf], (4,), float32)
Action space: Discrete(2)
Initial observation: [ 0.0262642   0.00729001 -0.01409904 -0.03187358]
```

Random Agent Test

Let's test the environment with a random agent to see how it performs.

In [3]:
```
while True:
    action = env.action_space.sample()
    obs, reward, done, truncated, info = env.step(action)
    total_reward += reward
    plt.imshow(env.render())
    display.display(plt.gcf())
    display.clear_output(wait=True)
    plt.close()

    if done or truncated:
        break

print(f"Random agent - Total reward: {total_reward}, Total steps: {total_ste
```

```
Random agent - Total reward: 10.0, Total steps: 0
```

Part 2: Linear Policy Implementation

We'll implement a simple linear policy that maps observations to actions using a weight matrix W and bias vector b.

In [4]:
```
class LinearPolicy(object):
    def __init__(self, theta, ob_space, ac_space):
        assert len(theta) == (ob_space + 1) * ac_space
        self.W = theta[0:ob_space*ac_space].reshape(ob_space, ac_space)
        self.b = theta[ob_space*ac_space:None].reshape(1, ac_space)

    def act(self, obs):
        y = obs.dot(self.W) + self.b
        a = y.argmax()
        return a


# Test the policy
ob_space = env.observation_space.shape[0]
ac_space = env.action_space.n
n_theta = (ob_space + 1) * ac_space
print(f"Policy parameter size: {n_theta}")

def run_episode(policy, env, num_steps, render=False):
    total_rew = 0
    ob, info = env.reset()
    for t in range(num_steps):
        a = policy.act(ob)
        ob, reward, done, truncated, info = env.step(a)
        total_rew += reward
        if done or truncated:
            break
    return total_rew


theta_mean = np.zeros(n_theta)
```

```
theta_std = np.ones(n_theta)

reward_list = []

# CEM hyperparameters
n_iterations = 50  # Number of CEM iterations
n_samples = 25     # Number of policy samples per iteration
n_elite = 5        # Number of elite samples to keep

print("Starting CEM training...")
print(f"Iterations: {n_iterations}, Samples: {n_samples}, Elite: {n_elite}")
print("-" * 60)

for itr in range(n_iterations):
    # Sample policies from current distribution
    thetas = np.random.multivariate_normal(
        mean=theta_mean,
        cov=np.diag(np.array(theta_std)**2),
        size=n_samples
    )

    rewards = []
    for theta in thetas:
        policy = LinearPolicy(theta, ob_space, ac_space)
        r = run_episode(policy, env, 500)
        rewards.append(r)

    rewards = np.array(rewards)

    # Get elite parameters
    elite_inds = rewards.argsort()[-n_elite:]
    elite_thetas = thetas[elite_inds]

    # Update theta_mean, theta_std
    theta_mean = elite_thetas.mean(axis=0)
    theta_std = elite_thetas.std(axis=0)

    # Log progress
    print(f"[Iteration {itr:2d}] mean: {np.mean(rewards):5.1f} | max: {np.ma
    reward_list.append(np.mean(rewards))

print("-" * 60)
print("Training complete!")

env.close()
```

```
Policy parameter size: 10
Starting CEM training...
Iterations: 50, Samples: 25, Elite: 5
------------------------------------------------------------
[Iteration  0] mean:  16.7 | max:  85.0 | min:   8.0
[Iteration  1] mean:  66.4 | max: 500.0 | min:   8.0
[Iteration  2] mean:  56.9 | max: 267.0 | min:  10.0
[Iteration  3] mean: 168.0 | max: 500.0 | min:  35.0
[Iteration  4] mean: 276.0 | max: 500.0 | min:   9.0
[Iteration  5] mean: 367.9 | max: 500.0 | min:  10.0
[Iteration  6] mean: 465.6 | max: 500.0 | min: 222.0
[Iteration  7] mean: 462.1 | max: 500.0 | min: 136.0
[Iteration  8] mean: 446.1 | max: 500.0 | min:  26.0
[Iteration  9] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 10] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 11] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 12] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 13] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 14] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 15] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 16] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 17] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 18] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 19] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 20] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 21] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 22] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 23] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 24] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 25] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 26] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 27] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 28] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 29] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 30] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 31] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 32] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 33] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 34] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 35] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 36] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 37] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 38] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 39] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 40] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 41] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 42] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 43] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 44] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 45] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 46] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 47] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 48] mean: 500.0 | max: 500.0 | min: 500.0
[Iteration 49] mean: 500.0 | max: 500.0 | min: 500.0
------------------------------------------------------------
Training complete!
```

Results Analysis - Discrete

What are the cons and pros of CEM for RL?

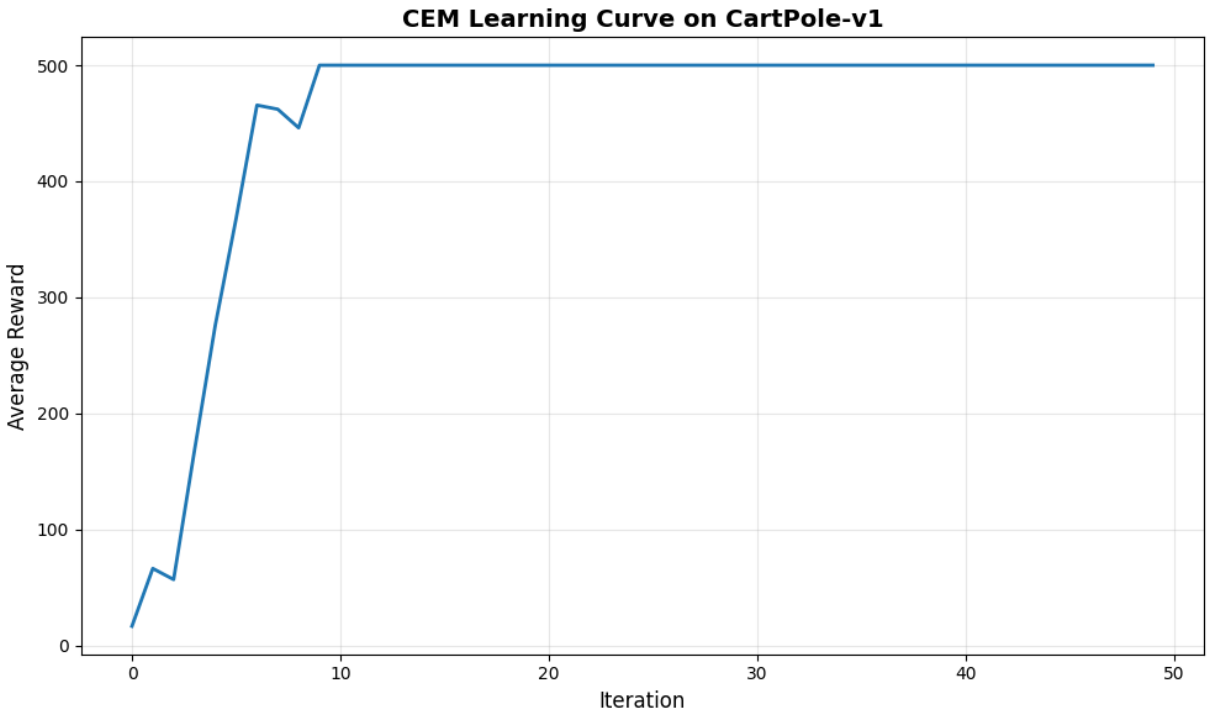
Pros: Models with no analytical gradient can be used. Also, since CEM is an exploratory genetic algorithm, the model is less likely to fall into a local minimum and is less sensitive to initial weights.

Cons: It's slow and expensive, since you must evaluate the model in the environment many thousands of times. If the model contains many parameters, it is unlikely that this probabilistic approach will randomly stumble upon the perfect combination of parameters, leading to poor convergence. A good use of this genetic algorithm could be to find reasonable initial values to start a gradient backpropgagation.

Picture of the learning curve of CEM on `CartPole-V1`:

In [5]:
```
# Plot learning curve
plt.figure(figsize=(10, 6))
plt.plot(reward_list, linewidth=2)
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Average Reward', fontsize=12)
plt.title('CEM Learning Curve on CartPole-v1', fontsize=14, fontweight='bold
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print statistics
print(f"Initial average reward: {reward_list[0]:.2f}")
print(f"Final average reward: {reward_list[-1]:.2f}")
print(f"Best average reward: {max(reward_list):.2f}")
print(f"Improvement: {reward_list[-1] - reward_list[0]:.2f}")
```




```
Initial average reward: 16.68
Final average reward: 500.00
Best average reward: 500.00
Improvement: 483.32
```

Save Results

Save the results to a CSV file for later analysis.

In [6]:
```
df = pd.DataFrame({"reward": reward_list})
df.to_csv("Linear_CEM.csv", index=False, header=True)

env.close()
```

Part 3: Linear Policy... but continuous!!

Use the same linear policy as in Part 3 but with a continuous output.

To do this we need the pendulum environment.

In [7]:
```
# Create pendulum environment
env2 = gym.make("Pendulum-v1", render_mode="rgb_array")

total_reward = 0.0
total_steps = 0
obs, info = env2.reset()

print(f"Observation space: {env2.observation_space}")
```

```
print(f"Action space: {env2.action_space}")
print(f"Initial observation: {obs}")
```

```
Observation space: Box([-1. -1. -8.], [1. 1. 8.], (3,), float32)
Action space: Box(-2.0, 2.0, (1,), float32)
Initial observation: [-0.08214566 -0.99662036  0.5801165 ]
```

In [20]:
```
# Make us a nice juicy continuous policy
class LinearContPolicy(object):
    def __init__(self, theta, ob_space, ac_space):
        assert len(theta) == (ob_space + 1) * ac_space
        self.W = theta[0:ob_space*ac_space].reshape(ob_space, ac_space)
        self.b = theta[ob_space*ac_space:None].reshape(1, ac_space)

    def act(self, obs):
        self.W = np.squeeze(self.W)
        obs = np.squeeze(obs)
        y = obs.dot(self.W) + self.b
        # a = y.argmax() # Instead of taking action 0 or 1 based on max prob
        # we shall squash the output to (-2,2).
        # I don't actually know what this -2 to 2 represents physically, but
        # a = ((1/(1 + np.exp(-y))) * 4) - 2 # Modified sigmoid
        a = np.tanh(y)*2
        return a
```

In [21]:
```
# Get continuous env observation and action spaces
ob_space = env2.observation_space.shape[0]
ac_space = env2.action_space.shape[0]
n_theta = (ob_space + 1) * ac_space
print(f"Size of continuous action space: {ac_space}")
print(f"Size of obs space: {ob_space}")
print(f"Policy parameter size: {n_theta}")
```

```
Size of continuous action space: 1
Size of obs space: 3
Policy parameter size: 4
```

(3 params for weight, 1 for bias)

In [22]:
```
# Run Cross-Entropy Method (genetic algorithm) on pendulum env
theta_mean = np.zeros(n_theta)
theta_std = np.ones(n_theta)

reward_list = []

# CEM hyperparameters
n_iterations = 50  # Number of CEM iterations
n_samples = 25     # Number of policy samples per iteration
n_elite = 5        # Number of elite samples to keep

print("Starting CEM training...")
print(f"Iterations: {n_iterations}, Samples: {n_samples}, Elite: {n_elite}")
print("-" * 60)

for itr in range(n_iterations):
    # Sample policies from current distribution
```

```
    thetas = np.random.multivariate_normal(
        mean=theta_mean,
        cov=np.diag(np.array(theta_std)**2),
        size=n_samples
    )

    rewards = []
    for theta in thetas:
        policy = LinearContPolicy(theta, ob_space, ac_space)
        r = run_episode(policy, env2, 500)
        rewards.append(r)

    rewards = np.array(rewards)
    rewards = np.squeeze(rewards)

    # Get elite parameters
    elite_inds = rewards.argsort()[-n_elite:]
    elite_thetas = thetas[elite_inds]

    # Update theta_mean, theta_std
    theta_mean = elite_thetas.mean(axis=0)
    theta_std = elite_thetas.std(axis=0)

    # Log progress
    print(f"[Iteration {itr:2d}] mean: {np.mean(rewards):5.1f} | max: {np.ma
    reward_list.append(np.mean(rewards))

print("-" * 60)
print("Training complete!")

env2.close()
```

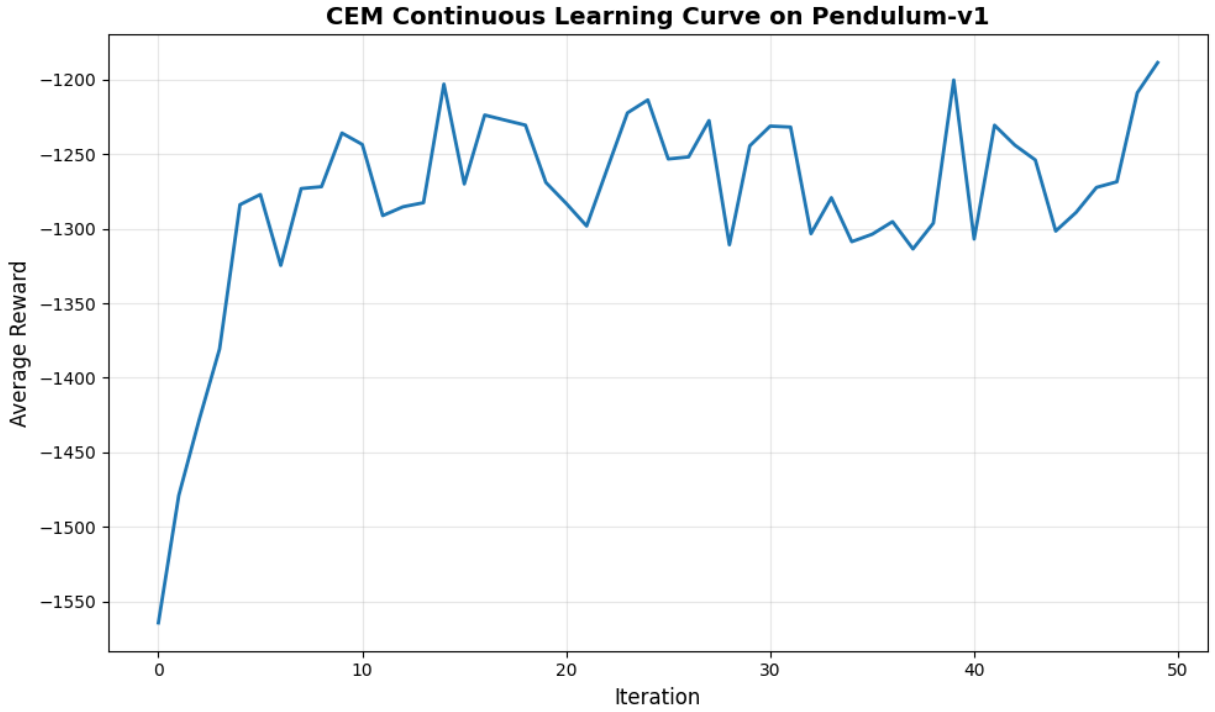
```
Starting CEM training...
Iterations: 50, Samples: 25, Elite: 5
------------------------------------------------------------
[Iteration  0] mean: -1564.3 | max: -1081.4 | min: -1854.9
[Iteration  1] mean: -1478.9 | max: -948.6 | min: -1841.6
[Iteration  2] mean: -1428.2 | max: -961.1 | min: -1834.0
[Iteration  3] mean: -1380.7 | max: -901.4 | min: -1600.7
[Iteration  4] mean: -1283.9 | max: -971.2 | min: -1526.2
[Iteration  5] mean: -1277.1 | max: -903.0 | min: -1510.6
[Iteration  6] mean: -1324.6 | max: -940.8 | min: -1517.2
[Iteration  7] mean: -1273.1 | max: -962.3 | min: -1506.9
[Iteration  8] mean: -1271.8 | max: -873.9 | min: -1519.0
[Iteration  9] mean: -1235.9 | max: -941.4 | min: -1516.6
[Iteration 10] mean: -1243.5 | max: -917.5 | min: -1500.7
[Iteration 11] mean: -1291.2 | max: -820.7 | min: -1525.0
[Iteration 12] mean: -1285.3 | max: -834.3 | min: -1529.2
[Iteration 13] mean: -1282.6 | max: -847.4 | min: -1514.3
[Iteration 14] mean: -1203.1 | max: -828.1 | min: -1476.7
[Iteration 15] mean: -1270.0 | max: -843.5 | min: -1502.7
[Iteration 16] mean: -1223.8 | max: -840.0 | min: -1502.9
[Iteration 17] mean: -1227.2 | max: -869.4 | min: -1515.0
[Iteration 18] mean: -1230.6 | max: -847.7 | min: -1496.2
[Iteration 19] mean: -1269.0 | max: -833.1 | min: -1507.3
[Iteration 20] mean: -1283.1 | max: -787.1 | min: -1500.2
[Iteration 21] mean: -1298.2 | max: -914.7 | min: -1516.3
[Iteration 22] mean: -1260.1 | max: -834.2 | min: -1509.5
[Iteration 23] mean: -1222.4 | max: -752.2 | min: -1512.1
[Iteration 24] mean: -1213.7 | max: -863.9 | min: -1496.4
[Iteration 25] mean: -1253.3 | max: -838.0 | min: -1518.1
[Iteration 26] mean: -1251.9 | max: -844.2 | min: -1511.5
[Iteration 27] mean: -1227.5 | max: -770.2 | min: -1499.6
[Iteration 28] mean: -1310.8 | max: -789.4 | min: -1511.5
[Iteration 29] mean: -1244.5 | max: -815.7 | min: -1504.1
[Iteration 30] mean: -1231.2 | max: -838.8 | min: -1522.0
[Iteration 31] mean: -1231.9 | max: -893.0 | min: -1498.2
[Iteration 32] mean: -1303.4 | max: -797.1 | min: -1505.5
[Iteration 33] mean: -1279.2 | max: -832.3 | min: -1511.1
[Iteration 34] mean: -1308.7 | max: -838.2 | min: -1527.0
[Iteration 35] mean: -1303.7 | max: -839.1 | min: -1525.1
[Iteration 36] mean: -1295.3 | max: -938.4 | min: -1512.1
[Iteration 37] mean: -1313.6 | max: -827.1 | min: -1517.8
[Iteration 38] mean: -1296.3 | max: -836.5 | min: -1521.2
[Iteration 39] mean: -1200.3 | max: -791.5 | min: -1505.2
[Iteration 40] mean: -1307.0 | max: -832.6 | min: -1524.8
[Iteration 41] mean: -1230.6 | max: -853.9 | min: -1494.0
[Iteration 42] mean: -1244.0 | max: -779.0 | min: -1516.9
[Iteration 43] mean: -1253.9 | max: -889.9 | min: -1499.5
[Iteration 44] mean: -1301.6 | max: -942.9 | min: -1521.0
[Iteration 45] mean: -1289.1 | max: -840.5 | min: -1523.2
[Iteration 46] mean: -1272.3 | max: -805.2 | min: -1517.9
[Iteration 47] mean: -1268.6 | max: -752.8 | min: -1527.6
[Iteration 48] mean: -1208.9 | max: -835.1 | min: -1499.3
[Iteration 49] mean: -1188.7 | max: -731.6 | min: -1498.0
------------------------------------------------------------
Training complete!
```

Results Analysis - Continuous

Let's visualize the learning curve and analyze the results.

In [23]:
```
# Plot learning curve
plt.figure(figsize=(10, 6))
plt.plot(reward_list, linewidth=2)
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Average Reward', fontsize=12)
plt.title('CEM Continuous Learning Curve on Pendulum-v1', fontsize=14, fontw
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print statistics
print(f"Initial average reward: {reward_list[0]:.2f}")
print(f"Final average reward: {reward_list[-1]:.2f}")
print(f"Best average reward: {max(reward_list):.2f}")
print(f"Improvement: {reward_list[-1] - reward_list[0]:.2f}")
```




```
Initial average reward: -1564.29
Final average reward: -1188.65
Best average reward: -1188.65
Improvement: 375.64
```

Gee, that didn't do too hot...

Save Results

Save the results to a CSV file for later analysis.

In [24]:
```
df = pd.DataFrame({"reward": reward_list})
df.to_csv("Linear_CEM_Continuous.csv", index=False, header=True)

env.close()
```

Part 4: Train a Silly "Neural Network" with a Genetic Algorithm

This one is kinda silly, but we may get better results!
Thought process:

1. The point of adding more and more neural network layers is to allow estimation of a complex nonlinear function.
2. Adding more and more neural network layers adds more parameters, making it difficult for a genetic algorithm to converge.
3. This system (`Pendulum_v1`) is nonlinear because it is a **circle**.
4. Including **circular** functions in the model may account for the nonlinearities of this system without adding a huge number of parameters!

This network (`KeviNet`) is essentially a linear policy and a "circular" policy. The linear policy is the same as the linear policy in the previous section. The circular policy is identical, except it wraps x within a cosine function:

$$input = cos(\omega * input + \phi)$$

where ω is some frequency and ϕ is some phase offset. Both are parameters to be trained, with the same dimensions as the input. Just basic physics 2 stuff.

In [ ]:
```
class KeviNet:
    def __init__(self, input_dim, output_dim):
        # Normal densely connected layer
        self.Wn = np.random.randn(input_dim, output_dim) * 0.1
        self.bn = np.zeros(output_dim)

        # Weird silly circular cosine layer
        self.omega = np.random.randn(input_dim) * 2*np.pi # To multiply by i
        self.phi = np.random.randn(input_dim) * 2*np.pi # Add to input*omega
        self.Wc = np.random.randn(input_dim, output_dim) * 0.1
        self.bc = np.zeros(output_dim)

        self.Wout = np.random.randn(output_dim, 2) * 0.1 # Combining the cir
        self.bout = np.zeros(output_dim)

        # How many theta?
        self.n_theta = self.Wn.size + self.bn.size + self.omega.size + self.

    def set_theta(self, theta):
        if(theta.size != self.n_theta):
```

```
            raise ValueError('Theta is the wrong size!')

        index = 0
        self.Wn = theta[index:index+self.Wn.size].reshape(self.Wn.shape)
        index += self.Wn.size
        self.bn = theta[index:index+self.bn.size].reshape(self.bn.shape)
        index += self.bn.size
        self.omega = theta[index:index+self.omega.size].reshape(self.omega.s
        index += self.omega.size
        self.phi = theta[index:index+self.phi.size].reshape(self.phi.shape)
        index += self.phi.size
        self.Wc = theta[index:index+self.Wc.size].reshape(self.Wc.shape)
        index += self.Wc.size
        self.bc = theta[index:index+self.bc.size].reshape(self.bc.shape)
        index += self.bc.size
        self.Wout = theta[index:index+self.Wout.size].reshape(self.Wout.shap
        index += self.Wout.size
        self.bout = theta[index:index+self.bout.size].reshape(self.bout.shap
        index += self.bout.size

    def get_n_theta(self):
        return self.n_theta

    def forward(self, x):
        # Transpose X to row vector for neural net purposes
        x = x.T

        # Normal densely connected layer
        zn = x @ self.Wn + self.bn
        an = np.tanh(zn)

        # Weird goofy nonlinear cosine layer
        zc = np.cos(x*self.omega + self.phi) @ self.Wc + self.bc
        ac = zc # It's a cosine so already -1 to 1

        # Combine the two weird layers
        intermediate = np.vstack((an, ac))
        zout = self.Wout @ intermediate + self.bout
        aout = np.tanh(zout)*2 # Without the activation function, the perfor

        return aout.T

    def act(self, x):
        return self.forward(x)
```

In [81]:

```
# Create pendulum environment
env2 = gym.make("Pendulum-v1", render_mode="rgb_array")

total_reward = 0.0
total_steps = 0
obs, info = env2.reset()

print(f"Observation space: {env2.observation_space}")
print(f"Action space: {env2.action_space}")
print(f"Initial observation: {obs}")
```

```
Observation space: Box([-1. -1. -8.], [1. 1. 8.], (3,), float32)
Action space: Box(-2.0, 2.0, (1,), float32)
Initial observation: [0.6168389 0.7870894 0.9321069]
```

In [82]:
```python
# Get continuous env observation and action spaces
ob_space = env2.observation_space.shape[0]
ac_space = env2.action_space.shape[0]
print(f"Size of continuous action space: {ac_space}")
print(f"Size of obs space: {ob_space}")

# Example network
net_ex = KeviNet(ob_space, ac_space)
n_theta = net_ex.get_n_theta()

print(f"Forward pass: {net_ex.forward(np.array([1,2,3]))}") # For debug
print(f"Policy parameter size: {n_theta}")
```

```
Size of continuous action space: 1
Size of obs space: 3
Forward pass: [[-0.09576011]]
Policy parameter size: 17
```

Note that we "only" have 17 parameters! (still a lot...)

In [110…
```python
# Run Cross-Entropy Method (genetic algorithm) on pendulum env with KeviNet
theta_mean = np.zeros(n_theta)
theta_std = np.ones(n_theta)*1

reward_list = []

# CEM hyperparameters
n_iterations = 200  # Number of CEM iterations
n_samples = 200     # Number of policy samples per iteration
n_elite_range = [20, 5]        # Number of elite samples to keep

print("Starting CEM training...")
print(f"Iterations: {n_iterations}, Samples: {n_samples}, Elite: {n_elite}")
print("-" * 60)

for itr in range(n_iterations):

    n_elite =  round(n_elite_range[0] - (n_elite_range[0] - n_elite_range[1]
    # Sample policies from current distribution
    thetas = np.random.multivariate_normal(
        mean=theta_mean,
        cov=np.diag(np.array(theta_std)**2),
        size=n_samples
    )

    rewards = []
    for theta in thetas:
        policy = KeviNet(ob_space, ac_space)
        policy.set_theta(theta=theta)
        r = run_episode(policy, env2, 500)
        rewards.append(r)
```

```
    rewards = np.array(rewards)
    rewards = np.squeeze(rewards)

    # Get elite parameters
    elite_inds = rewards.argsort()[-n_elite:]
    elite_thetas = thetas[elite_inds]


    # Update theta_mean, theta_std
    theta_mean = elite_thetas.mean(axis=0)
    theta_std = elite_thetas.std(axis=0)

    # Log progress
    print(f"[Iteration {itr:2d}] n_elite: {n_elite} mean: {np.mean(rewards):
    reward_list.append(np.mean(rewards))

print("-" * 60)
print("Training complete!")

env2.close()
```

```
Starting CEM training...
Iterations: 200, Samples: 200, Elite: 5
------------------------------------------------------------
[Iteration  0] n_elite: 20 mean: -1440.0 | max: -517.3 | min: -1936.4
[Iteration  1] n_elite: 20 mean: -1397.8 | max: -532.0 | min: -1834.3
[Iteration  2] n_elite: 20 mean: -1351.1 | max: -723.5 | min: -1747.6
[Iteration  3] n_elite: 20 mean: -1383.4 | max:  -3.2 | min: -1953.5
[Iteration  4] n_elite: 20 mean: -1346.4 | max: -662.7 | min: -1854.8
[Iteration  5] n_elite: 20 mean: -1273.5 | max: -523.9 | min: -1843.0
[Iteration  6] n_elite: 20 mean: -1274.6 | max: -135.3 | min: -1948.4
[Iteration  7] n_elite: 19 mean: -1242.4 | max:  -7.6 | min: -1920.8
[Iteration  8] n_elite: 19 mean: -1238.1 | max:  -3.7 | min: -1846.5
[Iteration  9] n_elite: 19 mean: -1196.0 | max: -600.1 | min: -1883.1
[Iteration 10] n_elite: 19 mean: -1188.5 | max: -358.8 | min: -1814.6
[Iteration 11] n_elite: 19 mean: -1208.0 | max: -500.5 | min: -1954.7
[Iteration 12] n_elite: 19 mean: -1181.0 | max:  -6.2 | min: -1957.8
[Iteration 13] n_elite: 19 mean: -1174.1 | max: -514.9 | min: -1831.0
[Iteration 14] n_elite: 19 mean: -1175.6 | max: -14.0 | min: -1962.6
[Iteration 15] n_elite: 19 mean: -1127.8 | max: -19.9 | min: -1862.2
[Iteration 16] n_elite: 19 mean: -1102.7 | max: -24.3 | min: -1966.1
[Iteration 17] n_elite: 19 mean: -1139.6 | max: -22.3 | min: -1947.3
[Iteration 18] n_elite: 19 mean: -1134.9 | max: -503.8 | min: -1899.6
[Iteration 19] n_elite: 19 mean: -1130.8 | max: -380.4 | min: -1787.4
[Iteration 20] n_elite: 18 mean: -1137.7 | max: -260.0 | min: -1825.8
[Iteration 21] n_elite: 18 mean: -1078.7 | max: -14.4 | min: -1711.6
[Iteration 22] n_elite: 18 mean: -1101.6 | max: -336.1 | min: -1773.7
[Iteration 23] n_elite: 18 mean: -1121.2 | max: -494.6 | min: -1730.1
[Iteration 24] n_elite: 18 mean: -1065.8 | max: -516.9 | min: -1687.3
[Iteration 25] n_elite: 18 mean: -1086.4 | max: -383.6 | min: -1723.8
[Iteration 26] n_elite: 18 mean: -1134.4 | max: -489.5 | min: -1692.2
[Iteration 27] n_elite: 18 mean: -1063.5 | max: -313.7 | min: -1712.8
[Iteration 28] n_elite: 18 mean: -1056.1 | max: -409.0 | min: -1664.7
[Iteration 29] n_elite: 18 mean: -1076.6 | max: -497.1 | min: -1678.2
[Iteration 30] n_elite: 18 mean: -1021.3 | max: -501.9 | min: -1664.7
[Iteration 31] n_elite: 18 mean: -1027.9 | max: -310.4 | min: -1663.2
[Iteration 32] n_elite: 18 mean: -1086.1 | max: -497.8 | min: -1676.3
[Iteration 33] n_elite: 18 mean: -1016.4 | max: -383.0 | min: -1658.6
[Iteration 34] n_elite: 17 mean: -1028.5 | max: -382.7 | min: -1693.4
[Iteration 35] n_elite: 17 mean: -1007.0 | max: -385.4 | min: -1686.2
[Iteration 36] n_elite: 17 mean: -1044.7 | max: -456.2 | min: -1685.2
[Iteration 37] n_elite: 17 mean: -1015.1 | max: -370.9 | min: -1680.9
[Iteration 38] n_elite: 17 mean: -993.6 | max: -381.3 | min: -1682.2
[Iteration 39] n_elite: 17 mean: -962.4 | max: -386.7 | min: -1678.3
[Iteration 40] n_elite: 17 mean: -971.5 | max: -382.0 | min: -1663.5
[Iteration 41] n_elite: 17 mean: -1008.8 | max: -376.9 | min: -1664.9
[Iteration 42] n_elite: 17 mean: -1047.1 | max: -466.6 | min: -1682.3
[Iteration 43] n_elite: 17 mean: -1011.6 | max: -384.7 | min: -1668.1
[Iteration 44] n_elite: 17 mean: -1000.5 | max: -384.5 | min: -1663.8
[Iteration 45] n_elite: 17 mean: -1029.2 | max: -385.7 | min: -1683.4
[Iteration 46] n_elite: 17 mean: -986.8 | max: -496.9 | min: -1664.6
[Iteration 47] n_elite: 16 mean: -997.7 | max: -384.7 | min: -1662.4
[Iteration 48] n_elite: 16 mean: -986.9 | max: -386.9 | min: -1667.0
[Iteration 49] n_elite: 16 mean: -972.3 | max: -408.0 | min: -1665.8
[Iteration 50] n_elite: 16 mean: -1006.0 | max: -475.6 | min: -1661.6
[Iteration 51] n_elite: 16 mean: -1002.7 | max: -391.5 | min: -1666.4
[Iteration 52] n_elite: 16 mean: -963.2 | max: -493.7 | min: -1666.3
```

```
[Iteration 53] n_elite: 16 mean: -1046.0 | max: -497.6 | min: -1662.7
[Iteration 54] n_elite: 16 mean: -998.1 | max: -394.5 | min: -1665.8
[Iteration 55] n_elite: 16 mean: -982.4 | max: -384.3 | min: -1666.1
[Iteration 56] n_elite: 16 mean: -999.4 | max: -479.7 | min: -1666.2
[Iteration 57] n_elite: 16 mean: -992.9 | max: -482.4 | min: -1667.0
[Iteration 58] n_elite: 16 mean: -978.0 | max: -382.4 | min: -1666.1
[Iteration 59] n_elite: 16 mean: -1010.8 | max: -381.7 | min: -1665.9
[Iteration 60] n_elite: 16 mean: -981.9 | max: -383.1 | min: -1662.3
[Iteration 61] n_elite: 15 mean: -989.2 | max: -382.9 | min: -1665.0
[Iteration 62] n_elite: 15 mean: -957.1 | max: -380.9 | min: -1667.5
[Iteration 63] n_elite: 15 mean: -940.0 | max: -382.1 | min: -1661.7
[Iteration 64] n_elite: 15 mean: -932.0 | max: -381.5 | min: -1669.7
[Iteration 65] n_elite: 15 mean: -990.8 | max: -379.5 | min: -1668.5
[Iteration 66] n_elite: 15 mean: -954.4 | max: -383.1 | min: -1667.1
[Iteration 67] n_elite: 15 mean: -956.1 | max: -384.0 | min: -1666.1
[Iteration 68] n_elite: 15 mean: -993.2 | max: -381.9 | min: -1666.9
[Iteration 69] n_elite: 15 mean: -946.0 | max: -383.7 | min: -1664.4
[Iteration 70] n_elite: 15 mean: -965.7 | max: -383.0 | min: -1661.8
[Iteration 71] n_elite: 15 mean: -956.6 | max: -382.2 | min: -1664.9
[Iteration 72] n_elite: 15 mean: -963.5 | max: -380.4 | min: -1660.0
[Iteration 73] n_elite: 15 mean: -946.7 | max: -383.8 | min: -1660.0
[Iteration 74] n_elite: 14 mean: -956.1 | max: -381.0 | min: -1669.8
[Iteration 75] n_elite: 14 mean: -979.6 | max: -382.9 | min: -1667.1
[Iteration 76] n_elite: 14 mean: -955.8 | max: -381.8 | min: -1665.8
[Iteration 77] n_elite: 14 mean: -923.0 | max: -382.6 | min: -1666.8
[Iteration 78] n_elite: 14 mean: -924.1 | max: -381.7 | min: -1666.5
[Iteration 79] n_elite: 14 mean: -927.2 | max: -379.6 | min: -1665.2
[Iteration 80] n_elite: 14 mean: -929.2 | max: -343.2 | min: -1664.7
[Iteration 81] n_elite: 14 mean: -946.7 | max: -382.3 | min: -1664.6
[Iteration 82] n_elite: 14 mean: -944.1 | max: -383.5 | min: -1664.0
[Iteration 83] n_elite: 14 mean: -961.1 | max: -380.2 | min: -1666.3
[Iteration 84] n_elite: 14 mean: -971.1 | max: -381.7 | min: -1665.6
[Iteration 85] n_elite: 14 mean: -936.9 | max: -383.1 | min: -1665.7
[Iteration 86] n_elite: 14 mean: -980.8 | max: -382.5 | min: -1665.1
[Iteration 87] n_elite: 13 mean: -933.9 | max: -382.2 | min: -1668.2
[Iteration 88] n_elite: 13 mean: -930.2 | max: -381.7 | min: -1666.6
[Iteration 89] n_elite: 13 mean: -922.3 | max: -382.7 | min: -1665.5
[Iteration 90] n_elite: 13 mean: -923.6 | max: -382.9 | min: -1662.2
[Iteration 91] n_elite: 13 mean: -961.2 | max: -381.0 | min: -1664.9
[Iteration 92] n_elite: 13 mean: -902.4 | max: -383.7 | min: -1663.2
[Iteration 93] n_elite: 13 mean: -956.7 | max: -383.0 | min: -1667.5
[Iteration 94] n_elite: 13 mean: -918.4 | max: -381.6 | min: -1666.9
[Iteration 95] n_elite: 13 mean: -938.2 | max: -382.5 | min: -1666.2
[Iteration 96] n_elite: 13 mean: -935.6 | max: -382.5 | min: -1668.4
[Iteration 97] n_elite: 13 mean: -913.0 | max: -383.2 | min: -1669.9
[Iteration 98] n_elite: 13 mean: -946.0 | max: -378.0 | min: -1663.4
[Iteration 99] n_elite: 13 mean: -952.6 | max: -381.6 | min: -1665.9
[Iteration 100] n_elite: 12 mean: -1015.4 | max: -382.6 | min: -1666.5
[Iteration 101] n_elite: 12 mean: -947.1 | max: -383.8 | min: -1665.4
[Iteration 102] n_elite: 12 mean: -904.4 | max: -381.0 | min: -1666.7
[Iteration 103] n_elite: 12 mean: -946.3 | max: -381.4 | min: -1664.7
[Iteration 104] n_elite: 12 mean: -976.9 | max: -382.5 | min: -1664.5
[Iteration 105] n_elite: 12 mean: -934.1 | max: -382.7 | min: -1662.7
[Iteration 106] n_elite: 12 mean: -918.4 | max: -382.3 | min: -1665.7
[Iteration 107] n_elite: 12 mean: -965.8 | max: -382.1 | min: -1665.5
[Iteration 108] n_elite: 12 mean: -942.5 | max: -373.8 | min: -1664.8
```

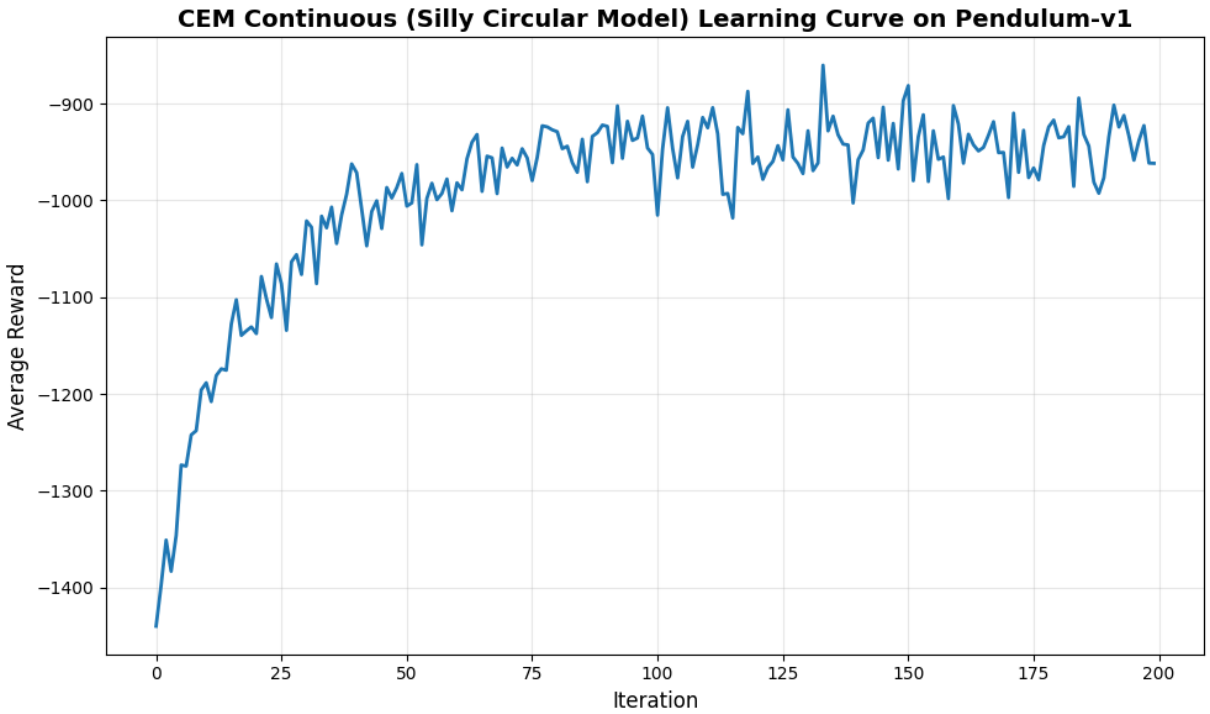
```
[Iteration 109] n_elite: 12 mean: -914.4 | max: -380.6 | min: -1669.1
[Iteration 110] n_elite: 12 mean: -925.2 | max: -382.8 | min: -1663.9
[Iteration 111] n_elite: 12 mean: -904.3 | max: -365.4 | min: -1668.2
[Iteration 112] n_elite: 12 mean: -931.4 | max: -382.0 | min: -1662.2
[Iteration 113] n_elite: 12 mean: -993.9 | max: -379.3 | min: -1666.7
[Iteration 114] n_elite: 11 mean: -992.9 | max: -382.6 | min: -1668.7
[Iteration 115] n_elite: 11 mean: -1018.3 | max: -382.9 | min: -1664.3
[Iteration 116] n_elite: 11 mean: -924.7 | max: -381.7 | min: -1658.7
[Iteration 117] n_elite: 11 mean: -931.2 | max: -383.1 | min: -1664.5
[Iteration 118] n_elite: 11 mean: -887.5 | max: -383.7 | min: -1665.1
[Iteration 119] n_elite: 11 mean: -962.0 | max: -382.4 | min: -1661.4
[Iteration 120] n_elite: 11 mean: -955.2 | max: -381.9 | min: -1663.3
[Iteration 121] n_elite: 11 mean: -978.4 | max: -382.5 | min: -1667.4
[Iteration 122] n_elite: 11 mean: -966.2 | max: -382.6 | min: -1665.3
[Iteration 123] n_elite: 11 mean: -959.8 | max: -382.8 | min: -1664.5
[Iteration 124] n_elite: 11 mean: -943.5 | max: -381.5 | min: -1660.7
[Iteration 125] n_elite: 11 mean: -958.3 | max: -382.7 | min: -1665.2
[Iteration 126] n_elite: 11 mean: -906.5 | max: -381.1 | min: -1666.0
[Iteration 127] n_elite: 10 mean: -955.3 | max: -383.0 | min: -1661.1
[Iteration 128] n_elite: 10 mean: -962.0 | max: -382.7 | min: -1665.7
[Iteration 129] n_elite: 10 mean: -972.4 | max: -382.3 | min: -1665.8
[Iteration 130] n_elite: 10 mean: -928.1 | max: -382.8 | min: -1664.3
[Iteration 131] n_elite: 10 mean: -969.4 | max: -342.9 | min: -1663.1
[Iteration 132] n_elite: 10 mean: -961.4 | max: -383.0 | min: -1655.9
[Iteration 133] n_elite: 10 mean: -860.5 | max: -382.5 | min: -1665.8
[Iteration 134] n_elite: 10 mean: -928.3 | max: -381.7 | min: -1667.5
[Iteration 135] n_elite: 10 mean: -913.1 | max: -381.7 | min: -1656.3
[Iteration 136] n_elite: 10 mean: -932.3 | max: -382.3 | min: -1664.2
[Iteration 137] n_elite: 10 mean: -941.9 | max: -280.7 | min: -1665.8
[Iteration 138] n_elite: 10 mean: -942.8 | max: -382.3 | min: -1659.0
[Iteration 139] n_elite: 10 mean: -1002.8 | max: -381.6 | min: -1668.4
[Iteration 140] n_elite: 10 mean: -958.1 | max: -381.7 | min: -1666.0
[Iteration 141] n_elite: 9 mean: -948.1 | max: -383.4 | min: -1665.0
[Iteration 142] n_elite: 9 mean: -920.2 | max: -383.7 | min: -1664.7
[Iteration 143] n_elite: 9 mean: -915.1 | max: -383.0 | min: -1664.7
[Iteration 144] n_elite: 9 mean: -956.0 | max: -383.7 | min: -1667.4
[Iteration 145] n_elite: 9 mean: -903.7 | max: -383.3 | min: -1657.7
[Iteration 146] n_elite: 9 mean: -958.5 | max: -382.5 | min: -1664.6
[Iteration 147] n_elite: 9 mean: -920.5 | max: -380.8 | min: -1661.5
[Iteration 148] n_elite: 9 mean: -967.7 | max: -383.3 | min: -1664.2
[Iteration 149] n_elite: 9 mean: -897.2 | max: -373.9 | min: -1665.7
[Iteration 150] n_elite: 9 mean: -881.5 | max: -381.2 | min: -1664.6
[Iteration 151] n_elite: 9 mean: -979.8 | max: -382.6 | min: -1667.6
[Iteration 152] n_elite: 9 mean: -934.3 | max: -382.7 | min: -1665.9
[Iteration 153] n_elite: 9 mean: -911.6 | max: -382.9 | min: -1660.2
[Iteration 154] n_elite: 8 mean: -980.6 | max: -382.5 | min: -1666.7
[Iteration 155] n_elite: 8 mean: -928.2 | max: -474.5 | min: -1664.7
[Iteration 156] n_elite: 8 mean: -957.7 | max: -382.5 | min: -1666.4
[Iteration 157] n_elite: 8 mean: -955.3 | max: -382.0 | min: -1666.3
[Iteration 158] n_elite: 8 mean: -998.4 | max: -380.3 | min: -1664.7
[Iteration 159] n_elite: 8 mean: -902.1 | max: -382.9 | min: -1663.7
[Iteration 160] n_elite: 8 mean: -921.2 | max: -383.0 | min: -1667.0
[Iteration 161] n_elite: 8 mean: -961.7 | max: -382.8 | min: -1667.0
[Iteration 162] n_elite: 8 mean: -931.7 | max: -380.7 | min: -1665.8
[Iteration 163] n_elite: 8 mean: -942.7 | max: -383.4 | min: -1667.2
[Iteration 164] n_elite: 8 mean: -949.0 | max: -382.0 | min: -1665.7
```

```
[Iteration 165] n_elite: 8 mean: -945.3 | max: -383.7 | min: -1663.9
[Iteration 166] n_elite: 8 mean: -932.7 | max: -382.9 | min: -1666.2
[Iteration 167] n_elite: 7 mean: -918.8 | max: -383.7 | min: -1668.6
[Iteration 168] n_elite: 7 mean: -950.8 | max: -380.4 | min: -1664.3
[Iteration 169] n_elite: 7 mean: -950.6 | max: -383.4 | min: -1667.9
[Iteration 170] n_elite: 7 mean: -997.2 | max: -382.5 | min: -1664.9
[Iteration 171] n_elite: 7 mean: -909.9 | max: -382.9 | min: -1664.4
[Iteration 172] n_elite: 7 mean: -971.2 | max: -382.1 | min: -1665.8
[Iteration 173] n_elite: 7 mean: -927.7 | max: -383.0 | min: -1665.5
[Iteration 174] n_elite: 7 mean: -976.6 | max: -384.0 | min: -1665.9
[Iteration 175] n_elite: 7 mean: -966.6 | max: -384.0 | min: -1666.7
[Iteration 176] n_elite: 7 mean: -978.8 | max: -382.6 | min: -1665.1
[Iteration 177] n_elite: 7 mean: -943.4 | max: -381.1 | min: -1661.4
[Iteration 178] n_elite: 7 mean: -924.2 | max: -378.2 | min: -1665.7
[Iteration 179] n_elite: 7 mean: -917.1 | max: -382.5 | min: -1667.2
[Iteration 180] n_elite: 6 mean: -935.5 | max: -382.4 | min: -1665.0
[Iteration 181] n_elite: 6 mean: -934.4 | max: -332.6 | min: -1664.3
[Iteration 182] n_elite: 6 mean: -923.8 | max: -381.6 | min: -1665.4
[Iteration 183] n_elite: 6 mean: -985.6 | max: -382.6 | min: -1667.2
[Iteration 184] n_elite: 6 mean: -894.3 | max: -383.1 | min: -1665.7
[Iteration 185] n_elite: 6 mean: -932.0 | max: -383.4 | min: -1663.5
[Iteration 186] n_elite: 6 mean: -943.9 | max: -379.7 | min: -1666.5
[Iteration 187] n_elite: 6 mean: -981.1 | max: -415.1 | min: -1668.0
[Iteration 188] n_elite: 6 mean: -992.8 | max: -380.6 | min: -1668.3
[Iteration 189] n_elite: 6 mean: -976.8 | max: -382.9 | min: -1665.6
[Iteration 190] n_elite: 6 mean: -934.5 | max: -380.4 | min: -1659.1
[Iteration 191] n_elite: 6 mean: -901.7 | max: -328.7 | min: -1665.8
[Iteration 192] n_elite: 6 mean: -924.3 | max: -382.9 | min: -1657.9
[Iteration 193] n_elite: 6 mean: -912.3 | max: -381.0 | min: -1658.5
[Iteration 194] n_elite: 5 mean: -933.5 | max: -383.6 | min: -1666.7
[Iteration 195] n_elite: 5 mean: -958.6 | max: -382.5 | min: -1663.7
[Iteration 196] n_elite: 5 mean: -938.3 | max: -380.8 | min: -1662.6
[Iteration 197] n_elite: 5 mean: -922.8 | max: -382.9 | min: -1665.7
[Iteration 198] n_elite: 5 mean: -961.5 | max: -377.1 | min: -1667.0
[Iteration 199] n_elite: 5 mean: -961.8 | max: -383.4 | min: -1666.8
------------------------------------------------------------
Training complete!
```

In [113…

```python
# Plot learning curve
plt.figure(figsize=(10, 6))
plt.plot(reward_list, linewidth=2)
plt.xlabel('Iteration', fontsize=12)
plt.ylabel('Average Reward', fontsize=12)
plt.title('CEM Continuous (Silly Circular Model) Learning Curve on Pendulum-
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print statistics
print(f"Initial average reward: {reward_list[0]:.2f}")
print(f"Final average reward: {reward_list[-1]:.2f}")
print(f"Best average reward: {max(reward_list):.2f}")
print(f"Improvement: {reward_list[-1] - reward_list[0]:.2f}")
```




```
Initial average reward: -1440.03
Final average reward: -961.81
Best average reward: -860.45
Improvement: 478.21
```

Save Results

Save the results to a CSV file for later analysis.

In [ ]:
```
df = pd.DataFrame({"reward": reward_list})
df.to_csv("Linear_CEM_Continuous_circular.csv", index=False, header=True)

env.close()
```